

DATA VORTEX — ROUND 1 PHASE 2

Final SQL Queries

Selected questions: E3 (Easy), M1 (Medium), H6 (Hard)

E3 — Average Engagement by Platform

```
-- E3 — Average Engagement by Platform
-- Calculate average likes, shares, comments and total engagement
-- for each platform.
-- Posts with missing likes are excluded from engagement averages.
```

```
SELECT
platform,
COUNT(*) AS post_count,
ROUND(AVG(likes), 2) AS avg_likes,
ROUND(AVG(shares), 2) AS avg_shares,
ROUND(AVG(comments), 2) AS avg_comments,
ROUND(AVG(engagement), 2) AS avg_total_engagement
FROM posts
WHERE likes IS NOT NULL
AND engagement IS NOT NULL
GROUP BY platform
ORDER BY avg_total_engagement DESC;
```

M1 — Which Locations Generate the Most Engagement?

```
-- M1 — Which Locations Generate the Most Engagement?
-- Join users and posts, aggregate engagement by location,
-- and rank locations from highest to lowest total engagement.
```

```
SELECT
u.location,
COUNT(p.post_id) AS post_count,
SUM(p.engagement) AS total_engagement
FROM users AS u
INNER JOIN posts AS p
ON u.user_id = p.user_id
WHERE p.engagement IS NOT NULL
GROUP BY u.location
ORDER BY total_engagement DESC;
```

H6 — Find the Most Suspicious High Impact Users

```
-- H6 — Find the Most Suspicious High Impact Users
--
-- Conditions:
-- 1. Fewer than 10,000 followers
-- 2. Average post engagement is above the overall average
-- 3. At least one post has more shares than likes
--
-- Missing engagement is excluded from engagement calculations.
```

```
WITH overall_average AS (
SELECT
AVG(engagement) AS overall_avg_engagement
FROM posts
WHERE engagement IS NOT NULL
),

user_metrics AS (
SELECT
u.user_id,
u.location,
u.follower_count,
COUNT(p.post_id) AS post_count,
AVG(p.engagement) AS avg_engagement,
SUM(p.engagement) AS total_engagement
FROM users AS u
INNER JOIN posts AS p
ON u.user_id = p.user_id
WHERE p.engagement IS NOT NULL
GROUP BY
u.user_id,
```

```

u.location,
u.follower_count
),

suspicious_users AS (
SELECT
um.user_id,
um.location,
um.follower_count,
um.post_count,
um.avg_engagement,
um.total_engagement
FROM user_metrics AS um
CROSS JOIN overall_average AS oa
WHERE um.follower_count < 10000
AND um.avg_engagement > oa.overall_avg_engagement
AND EXISTS (
SELECT 1
FROM posts AS p
WHERE p.user_id = um.user_id
AND p.likes IS NOT NULL
AND p.shares > p.likes
)
)

SELECT
user_id,
location,
follower_count,
post_count,
ROUND(avg_engagement, 2) AS avg_engagement,
total_engagement
FROM suspicious_users
ORDER BY total_engagement DESC;

```

All analytical outputs are generated dynamically from the SQLite database. No result values are hardcoded into the analytical queries.